Toxiro Taller LangGraph — Bitácora del día 1

Oct 20, 2026 · @ToxiroApps

LG-001 está implementado y probado en local; falta verificar la traza en LangSmith para aprobar el commit.

Estado actual

LG-001 está implementado en `C:\Desarrollo\toxiro-taller-LangGraph`. Los 4 tests pasan y `main.py` imprime el estado final completo. Git está inicializado, sin commits.

Para cerrar el ticket falta un solo paso, a cargo del Director: verificar la traza en LangSmith. Con eso, el CTO aprueba commit y push; falta el visto bueno del Director.

Roles: Rodrigo es Director, Claude (este proyecto) es CTO y Claude Code es Desarrollo. El CTO no habla con Desarrollo; todo pasa por Rodrigo.

Ajustes al plan original

1. **El checkpointer va antes que el interrupt.** Pausar el grafo significa guardar el estado y retomarlo con el mismo `thread_id`; sin persistencia no hay pausa posible.
2. **La trazabilidad entra desde LG-001, no al final.** LangSmith se activa con variables de entorno, sin tocar los nodos, y así cada ticket se ve en la traza mientras se construye. "Observabilidad desde el día cero" es una respuesta de arquitecto.
3. **Databricks Free Edition tiene límites que hay que verificar al inicio del día 2.** Corre solo sobre cómputo serverless e incluye Unity Catalog, Delta Lake y Lakeflow, pero su catálogo administrado tiene restricciones propias. La duda principal es el GRANT: con un solo usuario, demostrar permisos tiene gracia limitada.
4. **El servidor MCP probablemente no alcanza.** Queda como opcional al final.

Conceptos base

Estado. Es la ficha de la orden de trabajo que pasa de mano en mano: un objeto tipado único que todos los nodos leen y modifican. Cada nodo devuelve solo los campos que cambió y LangGraph los mezcla con el estado. Los **reductores** definen cómo se combinan escrituras sucesivas de un campo: por defecto el valor nuevo reemplaza al anterior; con un reductor de agregar, se acumula.

Nodos. Cada agente es una función que recibe el estado y devuelve una actualización parcial. El nodo no sabe qué viene después.

Bordes. El flujo vive fuera de los nodos, declarado en el grafo:

- **Borde normal:** después de A, siempre B.
- **Borde condicional:** una función lee el estado y elige el destino.
- **START y END:** entrada y salida.

Al final se **compila** el grafo: se valida la estructura y se conecta el checkpointer.

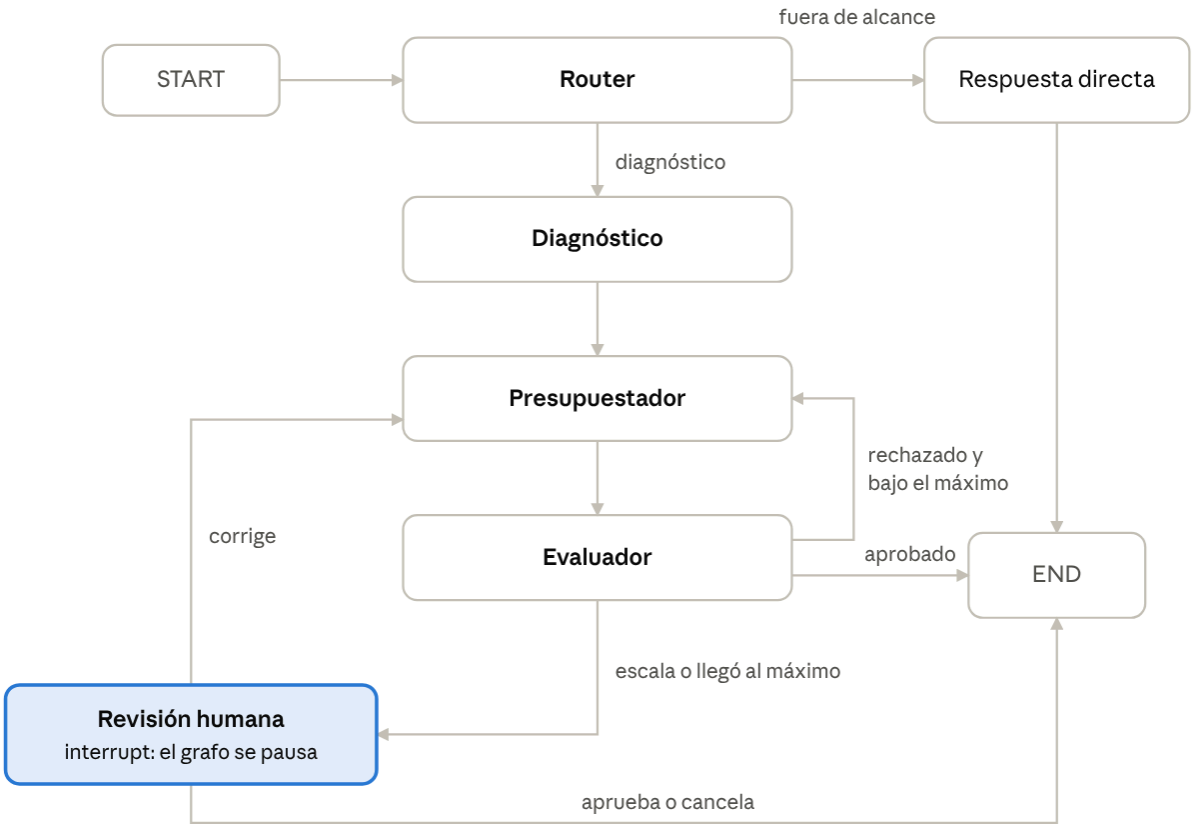
Borde condicional vs. `Command`

	Borde condicional	`Command` desde el nodo
Qué es	El nodo escribe en el estado y una función aparte decide el destino	El nodo devuelve actualización y destino juntos
Ventaja	El flujo es visible en la definición del grafo, fácil de dibujar y auditar	Menos piezas
Desventaja	Una función más por decisión	El flujo se esconde dentro de los nodos

Decisión: bordes condicionales. Para un cargo donde pesan trazabilidad y control, el flujo debe ser legible en un solo lugar.

Estructura del grafo

El Evaluador decide: aprobar, reintentar o pausar para revisión humana



grafo de Toxiro Taller · 4 agentes, bucle de corrección y revisión humana

El rechazo solo vuelve al Presupuestador; si el problema está en el diagnóstico, el Evaluador escala directo. La revisión humana pausa el grafo y lo retoma con la decisión del supervisor.

Campos del estado

Campo	Lo escribe	Nota
`sintoma`	Entrada	Texto del recepcionista
`intencion`	Router	Conjunto cerrado de opciones, no texto libre
`hipotesis`	Diagnóstico	Lista de {causa, probabilidad, urgencia}
`presupuesto`	Presupuestador	Ítems y totales
`evaluacion`	Evaluador	{veredicto, retroalimentación}
`retroalimentaciones`	Evaluador	Acumulativo, con reductor de agregar; solo en rechazos

Campo	Lo escribe	Nota
`intentos`	Evaluador	Cuenta rechazos
`decision_humana`	Revisión humana	Respuesta del supervisor

Decisiones tomadas

Decisión	Elegido	Razón
Destino del rechazo del Evaluador	Solo al Presupuestador	Simplicidad para el aprendizaje. Si el problema está en el diagnóstico, se escala directo a revisión humana sin reintentar: lo que un reintento no puede arreglar, no se reintenta
Tipo del estado	`TypedDict` para el estado, Pydantic para las salidas del LLM	La validación queda donde está el riesgo, en la respuesta del modelo
Quién incrementa `intentos`	El Evaluador	`intentos` cuenta rechazos. Con `MAX_REINTENTOS = 2`, el Presupuestador corre 3 veces en total
Máximo de reintentos	`MAX_REINTENTOS = 2`, configurable	Se usa desde LG-003
Nodos en LG-001	Simulados, sin LLM	Aprender estado, nodos y bordes sin el ruido del modelo
Ubicación de `TenantContext`	Contexto de ejecución, fuera del estado	No se copia en cada checkpoint ni en cada traza; separa configuración del cliente de los datos de ejecución. Se aplica desde LG-002
Ruteo	Bordes condicionales, no `Command`	Flujo legible y auditable en un solo lugar
Gestor de dependencias	`uv` (propuesto por Desarrollo)	Fija Python 3.12 por su cuenta y versiona un `uv.lock` reproducible

Decisión	Elegido	Razón
Estructura del repo	Paquete `taller/`, solo `main.py` en la raíz (propuesto por Desarrollo)	Evita choques de nombres genéricos con otras librerías

Mapa de tickets

Ticket	Día	Contenido
LG-001	1	Repo base, estado, grafo lineal con nodos simulados y LangSmith
LG-002	1	Router con borde condicional, LLM real (Haiku 4.5) con salida estructurada, `TenantContext` como contexto de ejecución
LG-003	1	Bucle Evaluador → Presupuestador con retroalimentación y máximo de reintentos
LG-004	1	Checkpointer (SQLite) y `thread_id`: retomar una conversación
LG-005	1	`interrupt` en revisión humana y reanudación
LG-006	2	Cuenta Free Edition, catálogo y esquema, tabla Delta de precios, Volume con chats simulados
LG-007	2	GRANT y linaje en Catalog Explorer, verificando qué permite Free Edition
LG-008	2	El Presupuestador consulta precios en Unity Catalog vía Databricks SQL
LG-009	2	Opcional: esa herramienta como servidor MCP

Cada día cierra con un resumen en palabras de arquitecto y 3 a 5 preguntas probables de entrevista.

Ticket-LG-001: repo base y grafo esqueleto

Objetivo: repo con estado tipado, 4 nodos simulados, grafo lineal START → router → diagnostico → presupuestador → evaluador → END, y trazabilidad en LangSmith. Fuera de alcance: LLM real, bordes condicionales, bucle, checkpointer, interrupt, FastAPI y Docker.

Resultado de Desarrollo

Criterio de aceptación	Estado
`main.py` imprime el estado final con los 4 campos poblados	Cumplido
Traza en LangSmith con los 4 nodos	Pendiente: falta la API key
Test del estado final (`tests/test_grafo.py`)	Cumplido
Test de que `retroalimentaciones` acumula (`tests/test_estado.py`)	Cumplido
Imagen del grafo con flujo lineal (`docs/grafo.png` , `docs/grafo.mmd`)	Cumplido, con test de las aristas

Versiones: Python 3.12.13, langgraph 1.2.16, langchain-anthropic 1.7.6, langsmith 0.14.11, pytest 9.1.1.

Cómo quedó:

- Estado en `taller/estado.py` , `TypedDict` con `total=False` ; `retroalimentaciones` es `Annotated[list[str], operator.add]` .
- `MAX_REINTENTOS` en `taller/config.py` , configurable por variable de entorno, todavía sin uso.
- Los tests desactivan `LANGSMITH_TRACING` . El `.env` está en `.gitignore` .
- Comandos: `uv sync` , `uv run pytest` , `uv run python main.py` , `uv run python -m taller.grafo` .
- Prueba del reductor en dos niveles: unitaria (el reductor sacado de la anotación del estado real) e integración mínima (dos nodos de prueba que agregan y se comprueba que acumula).

Revisión del CTO

- **Protocolo:** Desarrollo respondió las preguntas e implementó en el mismo paso, saltándose el acuerdo de los tres. En LG-001 no hizo daño; desde LG-002 debe entregar primero la revisión y esperar el visto bueno.
- **`total=False` :** aceptado. Efecto colateral: ningún campo es obligatorio, ni siquiera `sintoma` . Marcar `sintoma` como requerido en LG-002.
- **El Evaluador agrega retroalimentación incluso al aprobar:** corregir en LG-003. Solo se agrega cuando hay rechazo; si no, el Presupuestador leería una aprobación como corrección.

- **La imagen se genera con mermaid.ink, un servicio externo:** aceptable para aprender, porque solo viaja la estructura. En un entorno corporativo es un tema de gobernanza; suma mencionarlo en la entrevista.
- **Rango `langchain-anthropic>=1.7,<2`:** aprobado, como primer paso de LG-002.

Cierre pendiente (Director):

1. Crear cuenta en LangSmith y generar una API key.
2. Copiar `.env.example` a `.env` y poner la key. No pegarla en el chat.
3. Correr `uv run python main.py`.
4. En el proyecto `taller-lg-001`, revisar que la ejecución muestre los 4 nodos con su entrada y salida.

Comparación: orquestador propio vs. LangGraph

El detalle está en Comparación: orquestador propio vs. LangGraph — Toxiro Taller.

Tema	Orquestador propio hoy	LangGraph	Ganancia
Contexto	Objeto de caso que se enriquece, más `TenantContext` aparte; cada agente recibe todo	Estado tipado con actualizaciones parciales y reductores; `TenantContext` como contexto de ejecución	Contrato explícito: nadie pisa campos ajenos sin que se note
Reintento	Contador interno del orquestador, `feedback` en el caso, máximo como constante	`intentos` y `retroalimentaciones` en el estado; el bucle es un borde condicional	El control de flujo pasa a ser estado auditable, guardado y visible en la traza
Escalada	Marcador terminal (`status: escalated` + `escalation_package`); el supervisor no puede continuar el flujo	`interrupt` + checkpointer + `thread_id`: el grafo se pausa y se retoma con la decisión del supervisor	Resuelve un ticket que hoy está en la cola de pendientes
Quién decide el flujo	Código imperativo (`if` , `while`) en el orquestador	Bordes declarados en `grafo.py`	El flujo se compila, se valida, se dibuja y se ve en la traza

Límites de la escalada: LangGraph no da la integración con el panel ni con FastAPI; el endpoint que retoma el `thread_id` se escribe a mano. En producción el checkpointer debe ser durable (Postgres o similar).

Lo que se pierde: control total sobre cada línea, independencia de un proveedor y simplicidad en flujos lineales.

Regla: orquestador propio si el flujo es lineal, corto y sin intervención humana; LangGraph cuando hay que pausar, retomar, auditar el control de flujo o ramificar.

Autoevaluación

1. Un nodo devuelve solo `{"presupuesto": {...}}`. ¿Qué pasa con los demás campos?

- Tu respuesta: cada nodo devuelve solo los cambios que modificó.
- Corrección: correcta a medias; repite el supuesto y no dice qué pasa con el resto.
- Para la entrevista: *Los demás campos quedan intactos. LangGraph mezcla la actualización parcial con el estado existente y a cada clave devuelta le aplica su reductor, que por defecto es reemplazar.*

2. ¿Por qué `retroalimentaciones` necesita un reductor e `intencion` no?

- Tu respuesta: porque necesita saber que lo rechazaron.
- Corrección: eso explica para qué existe el campo, no por qué necesita reductor. El reductor define qué pasa cuando un campo se escribe más de una vez.
- Para la entrevista: *El historial de correcciones debe acumularse para que el Presupuestador no repita errores ya señalados; la intención es un valor puntual, donde el último gana.*

3. ¿Quién decide qué viene después y dónde está escrito?

- Tu respuesta: el borde normal.
- Corrección: correcta para LG-001, pero falta el quién y el dónde. Además, en tu orquestador no decide el agente sino el orquestador, con código imperativo.
- Para la entrevista: *Decide el grafo, no los nodos, declarado en `grafo.py`. Un borde normal fija el siguiente paso; uno condicional lee el estado y elige. A diferencia de mi orquestador, el flujo es una estructura declarada que se compila, se valida, se dibuja y se ve en la traza.*

Idea central: **el flujo pasa a ser un artefacto auditable**, que conecta con el control y la observabilidad que pide el cargo.

Pendientes

- [ ] Verificar la traza de LG-001 en LangSmith (Director)
- [ ] Aprobar commit y push de LG-001 (Director; el CTO ya aprobó, sujeto a la traza)
- [ ] Pedir a Desarrollo que desde LG-002 entregue la revisión antes de implementar
- [ ] Explicar el criterio con que el Evaluador decide hoy entre rechazar y escalar (necesario para LG-003 y LG-005)
- [ ] CTO: preparar la explicación previa y el Ticket-LG-002
- [ ] Verificar al inicio del día 2 qué permite Databricks Free Edition en GRANT y linaje

Correcciones agendadas:

- LG-002: fijar `langchain-anthropic>=1.7,<2`; marcar `sintoma` como requerido; `TenantContext` como contexto de ejecución.
- LG-003: el Evaluador solo agrega a `retroalimentaciones` cuando rechaza.